

# Zebra RFID + Impinj Gen2X Android Sample App

Android sample project for Zebra RFID + Impinj Gen2X features using the Zebra Android RFID SDK.

This README is aligned with the user guide in `ZebraGen2X_AppUserGuide.pptx` and the app implementation in this repository.

## Guide Coverage (from PPTX)

The user guide content covers: - Connect - Inventory - Singulation - Prefilter - TagFocus demo - Tag Quieting demo - Protected Mode demo - FastID demo

## What This App Does

- Discovers and connects Zebra readers.
- Configures RFID events and trigger mode.
- Streams read tags into UI.
- Supports Gen2X operations:
  - TagFocus
  - Tag Quiet / Unquiet
  - FastID
  - Protected Mode

## Main Files

- `app/src/main/java/com/example/newgen2xplay/MainActivity.java`
- `app/src/main/java/com/example/newgen2xplay/RFIDHandler.java`
- `app/src/main/java/com/example/newgen2xplay/ui/connect/ConnectFragment.java`
- `app/src/main/java/com/example/newgen2xplay/ui/Inventory/InventoryFragment.java`
- `app/src/main/java/com/example/newgen2xplay/ui/Inventory/InventoryViewModel.java`
- `app/src/main/java/com/example/newgen2xplay/ui/customImpinj/TagQuietCustomViewModel.java`

## Build

```
./gradlew :app:assembleDebug
```

Install:

```
./gradlew :app:installDebug
```

## Runtime Flow

1. `MainActivity` initializes `RFIDHandler`.
2. `ConnectFragment` refreshes and connects to a selected reader.
3. `RFIDHandler.connect(...)` connects, configures reader, and initializes `ImpinjExtensions`.
4. `RFIDHandler.EventHandler.eventReadNotify(...)` fetches tags and posts them to `tagDataViewModel`.
5. `InventoryFragment` observes tag data and updates `InventoryViewModel`.
6. Quiet/Unquiet and other feature operations are applied through `InventoryViewModel`.

## Detailed Code Snippets

### Reader connect and Impinj extension setup

```
if (!mConnectedRfidReader.isConnected()) {  
    mConnectedRfidReader.connect();  
    ConfigureReader();  
    if (mConnectedRfidReader.isConnected()) {
```

```

        impinjExtensions = new ImpinjExtensions(mConnectedRfidReader);
        connectionStatus.postValue(true);
    }
}

```

### Reader event configuration

```

mConnectedRfidReader.Events.addEventsListener(eventHandler);
mConnectedRfidReader.Events.setHandheldEvent(true);
mConnectedRfidReader.Events.setTagReadEvent(true);
mConnectedRfidReader.Events.setAttachTagDataWithReadEvent(false);
mConnectedRfidReader.Events.setReaderDisconnectEvent(true);

mConnectedRfidReader.Config.setTriggerMode(ENUM_TRIGGER_MODE.RFID_MODE, false);
mConnectedRfidReader.Config.setStartTrigger(triggerInfo.StartTrigger);
mConnectedRfidReader.Config.setStopTrigger(triggerInfo.StopTrigger);

```

### Tag data pipeline

```

TagData[] myTags = mConnectedRfidReader.Actions.getReadTags(100);
if (myTags != null) {
    context.runOnUiThread(() -> tagDataViewModel.setTagItems(myTags));
}

tagDataViewModel.getInventoryItem().observe(getViewLifecycleOwner(), tagItems -> {
    for (TagData data : tagItems) {
        inventoryViewModel.addOrUpdateInventoryItem(
            new InventoryItem(data.getTagID(), data.getTagSeenCount(), data.getPeakRSSI(), data.get...
        );
    }
});

```

### Quiet selected tags

```

mConnectedRfidReader.Actions.PreFilters.deleteAll();
setPrefilterForQuietingTags(items);
setTagQuietOrUnquiet(true);

impinjExtensions.setTagQuiet(
    new ENUM_TAGQUIET_MASK[] {ENUM_TAGQUIET_MASK.S3B},
    TARGET.TARGET_SL,
    STATE_AWARE_ACTION.STATE_AWARE_ACTION_ASRT_SL,
    (short) 1
);

```

### Unquiet tags

```

impinjExtensions.setTagQuiet(mask, TARGET.TARGET_SL, STATE_AWARE_ACTION.STATE_AWARE_ACTION_DSRT_SL, (short) 1);
impinjExtensions.setTagQuiet(mask, TARGET.TARGET_INVENTORIED_STATE_S3, STATE_AWARE_ACTION.STATE_AWARE_ACTION_DSRT_SL, (short) 1);

```

### Quiet timer behavior

```

String quietTimer = binding.quietTimer.getText().toString();
int quietTimeInMilliseconds = 2000;
if (!quietTimer.isEmpty()) {
    int seconds = Integer.parseInt(quietTimer);
}

```

```
    if (seconds > 0) quietTimeInMilliseconds = seconds * 1000;
}
quietHandler.postDelayed(this, quietTimeInMilliseconds);
```

## Documentation

- Guide extracted from slides: ZebraGen2X\_AppUserGuide.pptx
- App note: docs/AppNote-tag-quiet-timer-design.md
- Architecture + code review: design.md